

摘要

卷积神经网络 (CNN) 在计算机视觉领域取得了显著成就, 然而其庞大的计算量对嵌入式平台上的实时推理提出了严峻挑战。为满足边缘设备上目标检测算法的低延迟与低功耗需求, 本文面向 Zynq UltraScale+ XCK26 FPGA 平台, 设计了一款基于权重驻留脉动阵列的轻量级卷积神经网络 INT8 加速器, 目标应用为简化 YOLOv3-tiny 模型。

本文加速器采用 18×16 列的二维权重驻留脉动阵列作为核心计算单元, 利用 DSP48E2 的 $(A+D) \times B$ 运算技巧实现单 DSP 双 INT8 乘累加。架构上设计了五体行缓存与三维并行窗口提取器以支持步长和填充可配置的 3×3 卷积展开; 通过 K 维度分块、输出通道分块和空间分块三级分块策略, 支持任意规模卷积层的分时多轮累加; 引入乒乓缓冲的部分和反馈路径, 实现权重复用下的多轮累加语义。在系统接口层面, 设计并实现了 AXI-Lite 配置寄存器桥、64-bit AXI-Stream 四路数据搬运接口以及后处理流水线 (重量化、激活、HWC 地址写回), 形成完整的数据面与控制面闭环。

本文基于 Icarus Verilog 与 Vivado XSIM 构建了双引擎回归验证流程, 累计通过 60 余项模块级与系统级仿真测试。XCK26 综合结果表明, 18×16 配置下资源占用为 LUT 62%、DSP 28%、BRAM 31%, 完成布局布线后 100 MHz 核心时钟满足 $WNS = +0.26$ ns 的时序约束, 验证了该架构的工程可实现性。

关键词: 卷积神经网络, 脉动阵列, FPGA 加速, INT8 量化, AXI-Stream

目录

表格索引

插图索引

主要符号对照表

CNN	卷积神经网络 (Convolutional Neural Network)
IFM	输入特征图 (Input Feature Map)
OFM	输出特征图 (Output Feature Map)
PSUM	部分和 (Partial Sum)
DSP	数字信号处理单元 (Digital Signal Processing slice)
BRAM	块随机存取存储器 (Block RAM)
LUT	查找表 (Look-Up Table)
PE	处理单元 (Processing Element)
FSM	有限状态机 (Finite State Machine)
AXI	高级可扩展接口 (Advanced eXtensible Interface)
DMA	直接存储器访问 (Direct Memory Access)
K_{total}	展开后输入通道总维度, $K_{\text{total}} = C_{in} \times K_h \times K_w$
K_{TILE}	单次 K 分块处理的输入 lane 数
C_{out}	输出通道数
$\text{COUT}_{\text{TILE}}$	单次计算覆盖的输出通道数
H, W, C	特征图高度、宽度、通道数
K_h, K_w	卷积核高度、宽度
oy, ox	输出特征图空间坐标
GEMM	通用矩阵乘法 (GEneral Matrix Multiply)
FPGA	现场可编程门阵列 (Field Programmable Gate Array)
XCK26	Xilinx Kria K26 核心 FPGA 器件型号
WNS	最差负时序裕量 (Worst Negative Slack)
HWC	高度-宽度-通道 (Height-Width-Channel) 内存布局

第一章 绪论

1.1 研究背景与意义

近年来，卷积神经网络（Convolutional Neural Network, CNN）在图像分类、目标检测、语义分割等计算机视觉任务中取得了突破性进展。从 AlexNet、VGG 到 ResNet、EfficientNet，网络模型的深度和复杂度持续增长，检测精度不断提升。然而，高性能模型的参数量和计算量也随之急剧膨胀，这对资源受限的边缘嵌入式平台构成了严峻挑战。

在边缘计算场景中，无人机视觉、智能安防、自动驾驶等应用对实时目标检测提出了严格的延迟与功耗约束。若直接采用通用 CPU 或 GPU 部署 CNN 模型，前者计算吞吐能力不足，后者功耗过高难以满足嵌入式热约束。FPGA（Field Programmable Gate Array）以其可重构性、低功耗特性和丰富的片上 DSP 计算资源，成为边缘侧 CNN 推理加速的重要硬件平台。

YOLO（You Only Look Once）系列算法作为单阶段目标检测器的代表，在实时性方面具有天然优势。其中，YOLOv3-tiny 作为 YOLOv3 的轻量化版本，在保持一定检测精度的前提下大幅减少了参数量和计算量，是边缘端目标检测的理想选择。本文将针对简化版 YOLOv3-tiny 模型的卷积层计算瓶颈，基于 FPGA 平台设计一款高效的硬件加速器。

1.2 国内外研究现状

CNN 推理加速器的研究可根据计算架构大致分为单引擎卷积和脉动阵列两大类。

单引擎卷积架构以粗粒度算子（如 3×3 卷积核）为基本计算单元，通过循环展开和流水线并行完成卷积运算，代表性工作包括清华大学的 Angel-Eye^{guo2019angel} 等。此类方案控制逻辑相对简单，但对不同卷积参数（通道数、步长、填充）的灵活性受限。

脉动阵列架构自 H. T. Kung 于 1982 年提出以来^{kung1982systolic}，凭借其规整的二维网格互联和高效的权重/数据复用，在大规模矩阵乘法计算中展现了卓越的数据重用率和吞吐效率。Google 的 TPU^{jouppi2017tpu} 采用 256×256 的权重驻留脉动阵列，通过大规模二维乘法器网格实现了极高的推理吞吐。MIT 的 Eyeriss 系列^{chen2016eyeriss, chen2017eyerissv2} 则提出了行静态（Row Stationary）数据流，在能量效率方面做出了重要贡献。学术综述^{sz2017efficient} 对 DNN 加速器的数据流分类（权重驻留、输出驻留、输入驻留、行静态）进行了系统总结。

在 FPGA CNN 加速领域, 已有多个工作探索了脉动阵列或类脉动阵列架构。Qiu 等人的工作^{qiu2016going} 和 Ma 等人的工作^{ma2018scalable} 均可归入卷积加速模板化方法。Wei 等人^{wei2017automated} 提出了面向 FPGA 的自动化脉动阵列综合框架。然而, 多数方案仍在以下方面存在局限: (1) 对 INT8 细粒度量化的硬件原生支持不足; (2) 接口设计停留在仿真层面, 缺乏完整的 AXI-Stream 数据面闭环; (3) 未提供从模块级到系统级的完整 FPGA 综合与布线验证数据。

1.3 本文主要工作

针对上述问题, 本文面向 Xilinx Zynq UltraScale+ XCK26 平台, 设计并实现了一款基于权重驻留脉动阵列的轻量级 CNN INT8 加速器 IP, 主要工作包括:

1. **权重驻留脉动阵列核心:** 设计了 18 行 $\times$ 16 列的参数化二维脉动阵列。处理单元 (PE) 利用 Xilinx DSP48E2 的 $(A+D) \times B$ 运算模式实现 $\diamond\diamond$ 单 DSP 双 INT8 乘累加, 每列支持 2 个输出通道, 单阵列支持 32 路并行输出通道计算。
2. **完整数据流与分块策略:** 设计了 K 维度 (输入通道展开维度) 分块、输出通道 (Cout) 分块和空间 (Spatial) 分块三级分块策略, 支持任意规模卷积层的分时多轮累加。引入乒乓缓冲的部分和反馈路径, 实现权重复用下的多轮累加语义。
3. **后处理流水线:** 实现了 INT8 重量化模块、旁路/ReLU/Leaky ReLU 多模式激活模块和基于 HWC 地址的 OFM 写回模块, 形成完整的卷积层输出至外部存储的闭合链路。
4. **AXI 标准接口:** 设计了 AXI-Lite 配置寄存器桥, 以及 64-bit AXI-Stream 四路数据搬运接口 (偏置、权重、IFM 行、OFM 字节流), 支持 TVALID/TREADY/TLAST/TKEEP 全握手协议。
5. **系统验证与资源评估:** 基于 Icarus Verilog 与 Vivado XSIM 构建了双引擎回归验证流程, 累计通过 60 余项测试。在 XCK26 器件上完成了 18 $\times$ 16 配置的完整布局布线, 实现 100 MHz 时序收敛。

1.4 论文组织结构

本论文共分为六章, 组织结构如下:

第一章为绪论, 介绍 CNN 加速的研究背景、国内外研究现状及本文的主要贡献。

第二章为背景与相关工作, 介绍卷积运算基础、脉动阵列架构分类、INT8 量化原理、FPGA CNN 加速器相关工作以及目标平台 Zynq UltraScale+ XCK26 的基本特性。

第三章为加速器硬件架构设计, 重点阐述处理单元 (PE) 设计、二维脉动阵列结构、行缓存与窗口提取模块、权重加载与偏置注入机制、部分和反馈路径以及后处理

流水线的详细硬件实现。

第四章为数据流与分块策略,系统介绍卷积到分块 GEMM 的映射方法、K 分块/ Cout 分块/空间分块的三级分块调度、层调度有限状态机以及 PS/DMA 视角的软件调度契约。

第五章为接口设计与系统集成,介绍 AXI-Lite 配置寄存器接口设计、64-bit AXI-Stream 数据搬运接口的打包协议与实现、OFM 输出流 TLAST 生成逻辑以及 OFM 背压链设计。

第六章为验证与实验结果,报告 Icarus Verilog 和 Vivado XSIM 双回归的模块级与系统级验证结果,以及 32×32 、 32×16 、 16×16 、 18×16 四种配置在 XCK26 上的综合与布局布线资源对比,并对结果进行分析与讨论。

最后为总结与展望。

第二章 背景与相关工作

本章介绍与本文设计相关的基础理论与技术背景，包括卷积运算的数学定义及其 GEMM 转化、脉动阵列架构分类、INT8 量化原理、FPGA CNN 加速器相关工作以及目标硬件平台。

2.1 卷积神经网络与卷积运算

2.1.1 标准卷积运算

标准卷积层是 CNN 中最核心的计算算子。给定输入特征图 (Input Feature Map, IFM) $I \in \mathbb{R}^{C_{in} \times H \times W}$ 、卷积核 $W \in \mathbb{R}^{C_{out} \times C_{in} \times K_h \times K_w}$ 和偏置 $B \in \mathbb{R}^{C_{out}}$ ，输出特征图 (Output Feature Map, OFM) $O \in \mathbb{R}^{C_{out} \times H_{out} \times W_{out}}$ 的计算公式为：

$$O[c_{out}][y][x] = B[c_{out}] + \sum_{c_{in}=0}^{C_{in}-1} \sum_{k_y=0}^{K_h-1} \sum_{k_x=0}^{K_w-1} I[c_{in}][y \cdot S + k_y - P][x \cdot S + k_x - P] \cdot W[c_{out}][c_{in}][k_y][k_x] \quad (2.1)$$

其中， S 为步长 (Stride)， P 为填充 (Padding)， $H_{out} = (H + 2P - K_h)/S + 1$ ， $W_{out} = (W + 2P - K_w)/S + 1$ 。

YOLOv3-tiny 网络中主要使用 3×3 卷积 ($K_h = K_w = 3$) 和 1×1 卷积 (通道变换层)。本文加速器专门针对 3×3 卷积的硬件友好展开进行了优化， 1×1 卷积可作为 3×3 卷积的特例进行处理。

2.1.2 卷积到 GEMM 的映射

卷积运算可以通过 im2col (Image to Column) 变换转化为矩阵乘法 (GEMM) 形式。将 IFM 中与每个滑动窗口位置对应的 $C_{in} \times K_h \times K_w$ 个元素展开为一行，将 C_{out} 组卷积核分别展开为一列，即可将式 ?? 转化为：

$$OFM[p, c_{out}] = Bias[c_{out}] + \sum_{k=0}^{K_{total}-1} IFM[p, k] \cdot W[k, c_{out}] \quad (2.2)$$

其中， $p = oy \times W_{out} + ox$ 为输出像素的一维坐标， $K_{total} = C_{in} \times K_h \times K_w$ 为展开后的输入维度。这一映射将 6 层嵌套循环 (输出通道、输出高度、输出宽度、输入通道、卷积核高度、卷积核宽度) 约简为标准的矩阵乘法问题，极大便利了硬件加速器设计。

2.2 脉动阵列架构

2.2.1 基本概念

脉动阵列 (Systolic Array) 是一种由规整排列的处理单元 (Processing Element, PE) 组成的二维计算网格。数据以脉动方式在相邻 PE 间逐周期传递, 每个 PE 在每个时钟周期执行固定模式的乘加运算。脉动阵列的主要优点包括:

1. **规整性**: 所有 PE 结构相同, 适合 VLSI/FPGA 大规模阵列化实现。
2. **数据复用**: 权重、输入和部分和在阵列中以脉动传播, 大幅减少对全局存储的带宽需求。
3. **可扩展性**: 通过改变阵列行列数即可调整计算吞吐与资源开销的折中。

2.2.2 数据流分类

根据 Sze 等人^{sze2017efficient}的分类体系, 脉动阵列的数据流可按权重、输入和部分和的驻留策略分为以下四类:

1. **权重驻留 (Weight-Stationary)**: 每个 PE 存储一个或多个权重值, IFM 在行方向流动, PSUM 在列方向流动。代表作为 Google TPU^{jouppi2017tpu}。优点是权重从外部内存加载后驻留在 PE 中, 适合权重重用率较高的场景。
2. **输出驻留 (Output-Stationary)**: 每个 PE 负责累加一个输出像素的所有乘积累积, IFM 和权重在阵列中流动。优点是部分和驻留在 PE 内, 避免了 PSUM 的片外存储需求。
3. **输入驻留 (Input-Stationary)**: 每个 PE 存储一个 IFM 元素, 权重在阵列中流动。适合 IFM 数据量较小的场景。
4. **行静态 (Row-Stationary)**: Eyeriss^{chen2016eyeriss} 提出的数据流, 在 1D PE 行维度保持 IFM 元素驻留, 在 2D PE 列维度保持 filter 行元素驻留, 以最大化本地数据复用并最小化数据搬运能量。

本文选择权重驻留数据流, 原因如下: (1) YOLOv3-tiny 的 3×3 卷积核权重总量相对有限, 适合驻留在 PE 内部; (2) 权重驻留方案的控制逻辑更为简洁, 有利于 FPGA 实现中减少 LUT 开销; (3) 在 K 分块多轮累加场景下, 权重可在多个 PSUM 反馈轮次间保持驻留不变, 减少了权重重载次数。

2.3 INT8 量化

神经网络的量化是指将浮点权重和激活值映射到低位宽整数的过程。Jacob 等人提出的整数量化方案^{jacob2018quantization}将推理过程完全转换为整数运算:

$$q = \text{clamp} \left(\text{round} \left(\frac{r}{S} \right) + Z, q_{\min}, q_{\max} \right) \quad (2.3)$$

其中 r 为浮点值, S 为缩放因子 (Scale), Z 为零点 (Zero Point), $q_{\min}$ 和 $q_{\max}$ 为量化范围 (对于 INT8 为 -128 和 127)。

在推理时, 卷积计算完全以整数完成:

$$q_{out} = Z_{out} + \frac{S_w S_{in}}{S_{out}} \sum (q_w - Z_w)(q_{in} - Z_{in}) \quad (2.4)$$

其中, 预计算的乘数因子 $\frac{S_w S_{in}}{S_{out}}$ 可转化为定点乘加移位操作。本文加速器通过重量化 (Requantization) 模块, 以每一路输出通道一个 16-bit 乘数因子加 4-bit 移位量的参数化方式实现了此量化方案。

2.4 FPGA CNN 加速器相关工作

FPGA CNN 加速器的研究可追溯至 Qiu 等人^{qiu2016going}的工作, 该文首次系统性地探索了在嵌入式 FPGA 上部署 CNN 的完整设计流程。随后 Guo 等人在此基础上发布了 Angel-Eye 工具链^{guo2019angel}, 实现了从 Caffe 模型到 FPGA 比特流的自动化映射。

在脉动阵列 FPGA 实现方面, Wei 等人^{wei2017automated}提出了自动化脉动阵列综合框架, 支持从高层描述生成针对不同 FPGA 器件的 RTL 代码。Ding 等人^{ding2022tinyyolo}报道了一款面向 YOLOv3-tiny 的轻量级 FPGA CNN 加速器设计。然而, 这些工作大多侧重于高层设计空间探索或单一模块优化, 缺乏从 PE 级 DSP 原语到顶层 AXI-Stream 系统接口的端到端闭环设计和验证数据。

2.5 Zynq UltraScale+ XCK26 平台

本文目标平台为 Xilinx Kria K26 核心 FPGA 器件 XCK26 (具体型号 xck26-sfvc784-2LV-c), 属于 Zynq UltraScale+ MPSoC 系列。其主要逻辑资源如表 ?? 所示。

XCK26 的 DSP48E2 是一款高性能乘累加原语, 支持 27×18 -bit 乘法器和 48-bit 累加器^{xilinx2023dsp}。其关键特性之一是通过 $(A+D) \times B$ 的预加法运算模式, 可以在单个 DSP 内完成两项独立乘法的合并计算, 为本文的双 INT8 乘累加 PE 设计提供了硬件基础。

硬件平台的另一关键特性是完备的 AXI 互联体系。Zynq UltraScale+ 的处理系统 (Processing System, PS) 与可编程逻辑 (Programmable Logic, PL) 之间通过 AXI 接口实现高带宽低延迟的数据与控制交互^{arm2021axi,xilinx2023axi}。本文加速器设计的 AXI-Lite 配置接口和 AXI-Stream 数据接口即为后续 PS/DMA 协同调度的直接对接边界。

表 2.1 XCK26 (sfvc784-2LV-c) 主要逻辑资源

资源类型	可用数量
系统逻辑单元 (SLC)	约 256K
CLB LUT	117,120
CLB 触发器	234,240
Block RAM Tile	144 (36Kb 等效)
DSP48E2 Slice	1,248
用户 I/O	468
内部互联总线	AXI4-Lite / AXI4-Stream / AXI4-MM
PS 侧处理器	Quad-core ARM Cortex-A53

第三章 加速器硬件架构设计

本章详细阐述基于权重驻留脉动阵列的卷积加速器硬件架构设计，包括整体架构、处理单元（PE）设计、二维脉动阵列结构、行缓存与窗口提取、权重加载与偏置注入、部分和反馈路径以及后处理流水线。

3.1 整体架构概览

加速器顶层数据流遵循以下路径：DDR/DMA 通过 AXI-Stream 接口向行缓存注入 IFM 行数据；五体行缓存存储卷积窗口所需的输入特征图行；窗口提取器根据当前输出坐标 (oy, ox)、步长和填充并行提取 32 路 IFM 元素；IFM 通过 FIFO 以脉动偏斜时序注入二维脉动阵列；阵列中的每个 PE 驻留 2 个 INT8 权重，IFM 在行方向传播，PSUM 在列方向传播；阵列底部收集的 32 路 PSUM 经 PSUM FIFO 缓冲后，非最终 K 轮次的 PSUM 存入乒乓缓冲等待反馈，最终轮次的 PSUM 进入后处理流水线；后处理流水线包含重量化、激活和 OFM 写回三级，最终输出 INT8 OFM 字节流。

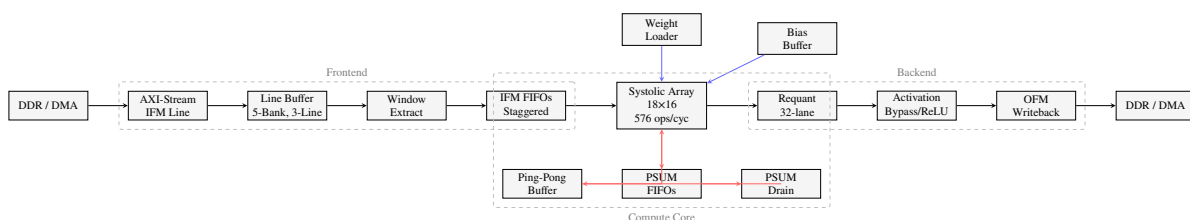


图 3.1 加速器整体架构框图

3.2 处理单元设计

处理单元（Processing Element, PE）是脉动阵列的基本计算节点，其硬件实现直接影响阵列的面积、吞吐和时序性能。

3.2.1 DSP48E2 双 INT8 乘法方案

Xilinx UltraScale+ 架构的 DSP48E2 原语内建 27×18 -bit 乘法器和 48-bit 累加器^{xilinx2023dsp}。为实现单 DSP 双 INT8 乘累加，本文利用 DSP48E2 的 $(A+D) \times B$ 预加法运算模式：

- 端口 A 载入 INT8 权重 a （符号扩展至 18 bit），端口 D 载入 INT8 权重 b （符号扩展至 18 bit）。
- 预加器计算 $A + D$ ，形成 27-bit 中间值。

- 端口 B 载入当前 IFM 值 c （符号扩展至 27 bit）。
- DSP 内部执行 $(A + D) \times B = A \times B + D \times B$ ，在同一乘法周期内获得 $ac = a \times c$ 和 $bc = b \times c$ 两项乘积。

当 b 为负数时需对 ac 结果进行符号校正，该逻辑在 DSP 旁路 LUT 中实现。整体计算管线包含 4 级流水：输入寄存、乘法、累加、输出寄存，在 100 MHz 时钟下每 DSP 每秒可完成 2 亿次 INT8 乘加操作。

3.2.2 PE 内部数据通路

每个 PE 内部包含以下关键路径：

- **IFM 水平传播链**：从左侧 PE 接收 IFM 值，寄存器一级后送往 DSP B 端口，同时寄存后传递至右侧 PE。水平方向 Valid 信号经 4 级流水延迟对齐计算管线。
- **权重驻留寄存器**：两个 INT8 权重值 w_a 和 w_b 分别锁存在 A 端口和 D 端口对应的寄存器中，在整个计算轮次（K Pass 或 Cout Block）内保持不变。
- **PSUM 垂直累加链**：从上方 PE 接收 PSUM 值（32-bit），与 DSP 输出的两项乘积求和后传递给下方 PE。垂直方向 Valid 信号经 5 级流水延迟。

表 3.1 单 PE 输入输出端口

信号	位宽	方向/含义
ifm_in / ifm_out	8 bit	水平 IFM 传播（左入右出）
psum_in / psum_out	32 bit	垂直 PSUM 传播（上入下出）
wgt_a / wgt_b	8 bit	双 INT8 权重驻留
valid_h_in / valid_h_out	1 bit	水平 Valid 传播
valid_v_in / valid_v_out	1 bit	垂直 Valid 传播

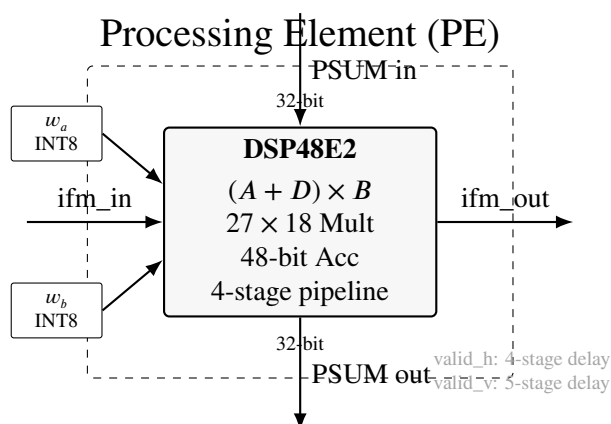


图 3.2 处理单元 (PE) 内部结构及 DSP48E2 双 INT8 数据通路

3.3 二维脉动阵列

3.3.1 阵列拓扑

二维脉动阵列由 $ROWS \times COLS$ 的 PE 网格组成, 主要配置为 $ROWS=18$ 、 $COLS=16$ 。阵列采用权重驻留数据流:

- **权重按列加载:** 每列的 18 个 PE 驻留来自 18 个不同输入 lane 的权重, 列内共享 2 个输出通道的计算。
- **IFM 按行脉动:** 阵列第 r 行的 16 个 PE 共享同一 IFM 输入值。第 r 行的 FIFO 读使能通过 $r \times 5$ 周期偏斜 (Stagger) 对齐阵列传播, 保证各 PE 在正确时刻接收到正确的 IFM 元素。
- **PSUM 按列累加:** PSUM 在列内从上向下传播并累加。第 0 行 (顶端) 接收外部注入的偏置或上一轮部分和; 第 $ROWS-1$ 行 (底端) 输出当前 K 轮次的累积结果。
- **PSUM 位宽:** 当前设定为 32 bit。YOLO 风格的大输入通道层存在 $C_{in} = 512/1024$ 的长累加链, 保守选择 32 bit 以避免溢出。

3.3.2 阵列吞吐分析

在当前 18×16 配置下, 阵列满流水时每时钟周期完成 $18 \times 16 \times 2 = 576$ 次 INT8 乘累加。在 100 MHz 核心时钟下等效峰值吞吐为 57.6 GOPS。作为对比, 若扩展至 32×32 配置 ($K_{TILE} = 32$, $COU_{TILE} = 64$), 理论峰值吞吐可达 $32 \times 32 \times 2 \times 100\text{MHz} = 204.8$ GOPS, 但 XCK26 的 LUT 资源不足以支撑该规模。

3.4 行缓存与窗口提取

3.4.1 五体行缓存

行缓存模块 (Line Buffer) 为窗口提取器提供 3×3 卷积所需的 3 行并行像素访问。其设计要点如下:

- **存储结构:** 5 Bank $\times$ 3 物理 Line。每条物理 Line 包含输入特征图一整行的像素数据, 采用环形替换策略 (最老行被最新填充行覆盖)。
- **Bank 交错:** 物理 Line 的 x 坐标像素分布于 5 个 Bank, Bank 索引 $= x \bmod 5$ 。对于 YOLOv3-tiny 通常的 5 通道输入, 此交错策略可避免 3×3 窗口内 Bank 访问冲突。当目标网络输入通道数较小时, Bank 数可相应缩减为 2。
- **三路并行读出:** 每条物理 Line 支持 $k_x = 0$, $k_x = 1$, $k_x = 2$ 三个独立读地址, 对应 3×3 卷积窗口的水平三列同时展开。

- **填充接口**: 通过 DMA 侧行填充服务驱动写入, 协议为 $feeder_fill_req + feeder_fill_f_y$ 发起请求, 外部按 $x = 0..fm_w-1$ 顺序每拍发送 5 个 Bank 字节, 以 $dma_line_advance$ 完成一行提交。

3.4.2 窗口提取

窗口提取器 (Window Extract) 负责将当前输出坐标 (oy, ox) 映射为 32-lane 的 IFM 向量, 供脉动阵列消费:

- 根据 (oy, ox) 、stride 和 pad 计算 3×3 窗口中 9 个位置的 IFM 空间坐标 (f_{y_i}, f_{x_i}) 。
- 从 Line Buffer 并行读出 3 行 $\times$ 3 列对应的像素值, 按 $k = c_{in} \times 3 \times 3 + k_y \times 3 + k_x$ 的展开顺序映射到 32 个并行 lane。
- 越界或填充位置的 lane 输出为 0。
- 通过检查 Line Buffer 的行有效标记 ($line_valid$) 和 f_y 匹配 (当前窗口需要的行已全部就位) 判定窗口是否就绪 (Window Ready)。未就绪时暂停 ox/oy 推进。

3.5 权重加载与偏置注入

权重加载模块 (Weight Tile Loader) 按 $K_{TILE} \times COUT_{TILE}$ 规模将权重 Tile 载入阵列中各 PE 的权重寄存器。加载顺序为:

$$\text{for } k_lane = 0..K_{TILE} - 1 : \text{ for } cout_lane = 0..COUT_{TILE} - 1 : w[k_lane][cout_lane] \quad (3.1)$$

外部以 64-bit AXI-Stream 发送, 每 beat 打包 8 个 INT8 权重, TLAST 在一个权重 Tile 的最后一 beat 拉高。对于当前典型 $K_{TILE} = 18, COUT_{TILE} = 32$ 的配置, 每次权重加载需要 $18 \times 32 / 8 = 72$ beats。

偏置 (Bias) 通过偏置缓冲模块向每列 PE 顶部注入。32-bit 偏置值按 $cout_lane = 0..COUT_{TILE} - 1$ 顺序注入, 仅在每个 Cout Block 的第一个 K Pass 时注入。后续 K Pass 以 PSUM 流注入替代偏置注入。

3.6 部分和反馈路径

PSUM 乒乓缓冲 (Ping-Pong Buffer) 是 K Pass 之间部分和数据的桥接模块。其工作原理为:

- PSUM 排出模块 (PSUM Drain Writer) 从阵列底部的 32 路 PSUM FIFO 中读取数据, 每个像素产生一个 $COUT_{TILE}$ 宽的 PSUM 包。

- 非最终 K Pass 的 PSUM 包写入乒乓缓冲的写半区。
- 下一 K Pass 开始时, PSUM 流注入模块 (PSUM Stream Feeder) 从乒乓缓冲的读半区读出 PSUM 包, 以倾斜时序 (Staggered) 注入阵列顶部对应列。
- 最终 K Pass 的 PSUM 包不经过乒乓缓冲, 直接推入后处理流水线。

乒乓双半区的设计使得一 K Pass 的 PSUM 排空 (Drain) 可以与下一 K Pass 的 PSUM 注入 (Feed) 在时间上重叠, 减少流水线气泡。

3.7 后处理流水线

3.7.1 重量化

重量化模块 (Requant) 将 32-bit PSUM 值转化为 INT8 结果。每路输出通道独立配置量化参数:

$$\text{OFM}_{int8}[c] = \text{clamp}_{-128}^{127} \left(\lfloor (\text{PSUM}[c] \times \text{mult}[c]) \gg \text{shift}[c] \rfloor + \text{zp}[c] \right) \quad (3.2)$$

硬件实现为 4 级流水线: 符号扩展乘法 (32-bit $\times$ 16-bit)、舍入加法、算术右移、零点加法与钳位。对于 $\text{COUT}_{TILE} = 32$ 的配置, 实例化 32 路并行重量化通道。

3.7.2 激活

激活模块支持三种模式:

- **Bypass** (mode=0): 直通输出。
- **ReLU** (mode=1): $\text{ReLU}(x) = \max(0, x)$, 以组合逻辑中的符号位检测实现。
- **Leaky ReLU** (mode=2): $x \geq 0$ 时输出 x , $x < 0$ 时输出 $\text{LUT}[|x|]$ 。查找表为 256-entry $\times$ 8-bit 的单端口 ROM。

3.7.3 OFM 写回

OFM 写回模块 (OFM Writeback) 将 COUT_{TILE} 宽的 INT8 OFM 包展开为字节流, 按 HWC 布局生成全局地址:

$$\text{wr_addr} = (\text{tile_pixel_base} + \text{local_pixel}) \times \text{cout_total} + (\text{cout_base} + \text{lane}) \quad (3.3)$$

其中 tile_pixel_base 为空间分块的全局像素基址, local_pixel 为 Tile 内像素序号, cout_base 为当前 Cout Block 的输出通道基址, lane 为并行通道编号。通过地址空间隔离实现 Spatial Tile 和 Cout Block 的独立写回。

3.8 模块层次封装

为支持从本地寄存器接口到标准总线的渐进集成，加速器核心通过 5 层封装逐步添加接口功能，如表 ?? 所示。每层封装均保持核心计算链路不变。

表 3.2 加速器顶层封装层次

模块	接口特征
conv_accel_core	本地 cfg_* 配置总线 + 直接数据端口
conv_accel_core_axi_lite	+ AXI-Lite 配置桥
conv_accel_core_axi_lite_stream	+ bias/weight ready-valid 流接口
conv_accel_core_axi_lite_full_stream	+ IFM/OFM 流接口（全四路）
conv_accel_core_axi_lite_axis_stream	+ AXI-Stream 64-bit 标准协议 (TVALID/TREADY/TLAST/TKEEP)

第四章 数据流与分块策略

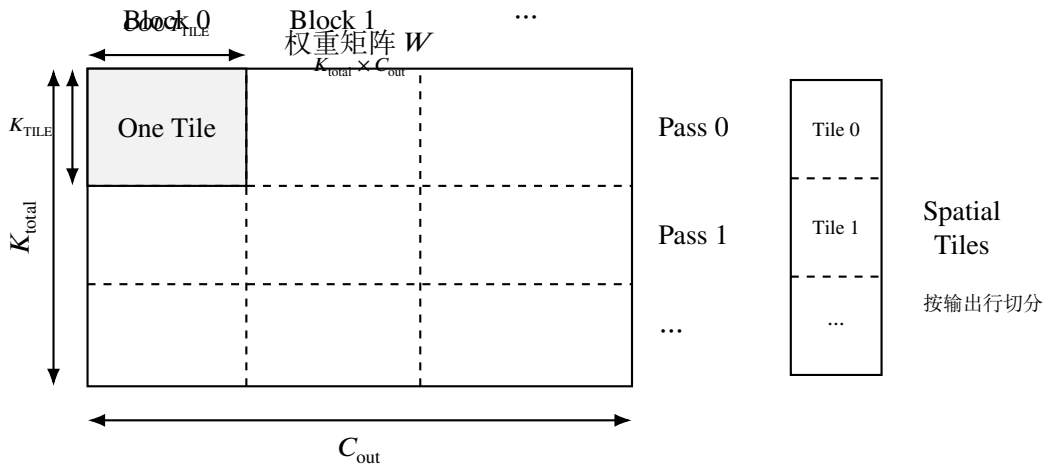
本章系统阐述加速器的数据流组织和三级分块调度策略，包括卷积到分块 GEMM 的映射方法、K 分块/Cout 分块/Spatial 分块的三级调度、层调度有限状态机以及 PS/DMA 调度契约。

4.1 卷积到分块 GEMM 的映射

如第 ?? 章所述，卷积运算可经 im2col 展开转化为 GEMM 形式。然而对于 YOLOv3-tiny 中的典型卷积层（如 $C_{in} = 512$, $C_{out} = 1024$, $K_h = K_w = 3$ ），展开后的矩阵维度 $K_{total} \times C_{out}$ 可达 4608×1024 ，远超脉动阵列的物理规模（ $K_{TILE} = 18$, $COU_{TILE} = 32$ ）。因此，本文采用三级分块策略将大规模 GEMM 分解为可装入阵列的子矩阵序列：

$$OFM[p, c_{out}] = \text{Bias}[c_{out}] + \sum_{p=0}^{K_p-1} \sum_{k=0}^{K_T-1} \text{IFM}[p, pK_T + k] \cdot W[pK_T + k, c_{out}] \quad (4.1)$$

分块的三个维度分别为：（1）K 维度——将 K_{total} 个展开 IFM lane 按 K_{TILE} 粒度切分；（2）Cout 维度——将输出通道按 COU_{TILE} 粒度切分；（3）Spatial 维度——将 OFM 空间像素按输出行范围切分。



K 分块：沿纵向切分，多 Pass 累加 Cout 分块：沿横向切分，逐 Block 替换权重 Spatial 分块：按输出行范围切分

图 4.1 三级分块策略示意图：K 维度、Cout 维度与 Spatial 维度

4.2 K 分块多轮累加

K 分块将 $K_{total} = C_{in} \times K_h \times K_w$ 个展开输入维度切分为 $K_passes = \lceil K_{total}/K_{TILE} \rceil$ 个连续子块，每个 K Pass 在一个 $K_{TILE} \times COUT_{TILE}$ 子矩阵上完成乘加运算。K Pass 间的 PSUM 通过乒乓缓冲传递：

1. **Pass #0**：注入 Bias 作为初始累加值，计算第一组 K_{TILE} 个输入 lane 的乘加。PSUM 经 Drain 后存入乒乓缓冲。
2. **Pass #1 至 Pass #(K_passes-2)**：从乒乓缓冲读取上轮 PSUM 注入阵列顶部作为初始值，继续累加当前 K 子块。结果写回乒乓缓冲。
3. **最终 Pass**：从乒乓缓冲读取上轮 PSUM，执行最后 K_{TILE} 个输入 lane 的乘加。结果直接发送至 Requant $\rightarrow$ Activation $\rightarrow$ Writeback 后处理链路。

当 K_{total} 不是 K_{TILE} 的整倍数时，最后 K Pass 仅包含 $K_{total} \bmod K_{TILE}$ 个有效输入 lane，其余 lane 的 IFM 输入恒为零，PSUM 仅加上零增量后继续向下传播。

对于典型的 YOLOv3-tiny 中间层 ($C_{in} = 16, K_h = K_w = 3$)， $K_{total} = 144$ 。以 $K_{TILE} = 32$ 为例需要 5 个 K Pass，以 $K_{TILE} = 18$ 为例需要 8 个 K Pass。K Pass 数与延迟成线性正比，与硬件资源（阵列行数）成反比。

4.3 输出通道 (Cout) 分块

当 $C_{out} > COUT_{TILE}$ 时，输出通道按 $COUT_{TILE}$ 粒度切分为连续块。Cout 分块的处理逻辑如下：

1. 对每个 $cout_base \in \{0, COUT_{TILE}, 2 \cdot COUT_{TILE}, \dots\}$ ，加载对应权重子矩阵 ($K_{TILE} \times COUT_{TILE}$)。
2. 遍历所有 K Pass 完成该 Cout Block 的完整累加。
3. 在最终 Pass 后，OFM 写回时以 $(cout_base + lane)$ 计算写回通道地址，不同 Cout Block 写到同一像素的不同通道范围。

关键优化：IFM 窗口数据在不同 Cout Block 间完全复用。这意味着对于同一 Spatial Tile 内的同一组 IFM 行，不需要重复窗口提取和 IFM 加载，仅需重新加载权重和执行累加。这显著节省了 IFM 带宽。

对于 YOLOv3-tiny 最后层 ($C_{out} = 1024$)，以 $COUT_{TILE} = 64$ 需 16 个 Cout Block，每个 Block 均需重新加载权重 tile 并完成全部 K Pass 循环。

4.4 空间 (Spatial) 分块

对于大尺寸特征图，片上 PSUM 缓冲容量和行缓存深度不足以容纳完整 OFM。空间分块将 OFM 按输出行切分为多个 Spatial Tile。

空间分块的四个配置参数为：

- `tile_oy_base`: Tile 在全局 OFM 的起始输出行。
- `tile_ofm_h`: Tile 覆盖的输出行数 (0 表示完整 OFM 高度)。
- `tile_pixel_base`: Tile 在 HWC OFM 中的全局像素起始下标, 通常 $= \text{tile_oy_base} \times \text{OFM_W}$ 。
- `num_pixels`: Tile 输出像素总数, 通常 $= \text{tile_ofm_h} \times \text{OFM_W}$ 。

OFM 写回的全局地址计算公式 (详见第 ?? 章) 保证了不同 Spatial Tile 的 OFM 数据在全局内存中按地址范围拼接, 无需 PS 侧做额外重组。

以 8×8 OFM 分为 3 个 Spatial Tile 的典型场景为例：

- Tile #0: 行 0-2, 覆盖 $3 \times 8 = 24$ pixels, `tile_pixel_base` = 0。
- Tile #1: 行 3-5, 覆盖 $3 \times 8 = 24$ pixels, `tile_pixel_base` = 24。
- Tile #2: 行 6-7, 覆盖 $2 \times 8 = 16$ pixels, `tile_pixel_base` = 48。

4.5 层调度有限状态机

层调度器 (Layer Scheduler Stream) 以有限状态机 (FSM) 协调三级分块的完整执行流程。FSM 的主要状态及转移关系如下：

1. **IDLE**: 等待外部写入 CTRL.start 脉冲。start 置位后跳转至 BIAS_LOAD。
2. **BIAS_LOAD**: 发起 bias 加载请求, 等待偏置流传输完成。完成后跳转至 WEIGHT_LOAD。
3. **WEIGHT_LOAD**: 发起权重加载请求, 等待权重 Tile 传输完成。完成后跳转至 LINE_FILL。
4. **LINE_FILL**: 根据当前 oy 范围和 K Pass 的展开基地址向 Window Feeder 发出行填充请求。收到 Line Buffer 的窗口就绪信号后跳转至 COMPUTE。
5. **COMPUTE**: 使能阵列 IFM FIFO 读, 启动窗口流推进。num_pixels 个像素全部流过后跳转。若非最终 K Pass 则跳转至 PSUM_DRAIN; 若为最终 K Pass 则跳转至 FINAL_DRAIN。
6. **PSUM_DRAIN**: 从 PSUM FIFO 排出 num_pixels 个 PSUM 包存入乒乓缓冲。完成后更新 K Pass 计数器, 跳转回 WEIGHT_LOAD 执行下一 K Pass。
7. **FINAL_DRAIN**: 将最终 PSUM 包排出并推入后处理流水线 (Requant $\rightarrow$ Activation $\rightarrow$ Writeback)。完成所有像素后检查 Cout Block 和 Spatial Tile 的循环状态：

- 若还有未处理的 Cout Block，更新 *cout_base* 跳转回 BIAS_LOAD。
 - 若所有 Cout Block 已完成且还有未处理的 Spatial Tile，跳转至 DONE。
 - 若所有 Tile 完成，跳转至 IDLE。
8. **DONE**: 置位 STATUS.done_sticky 标志，等待外部写入 CTRL.clear_done 清除后跳转至 IDLE。

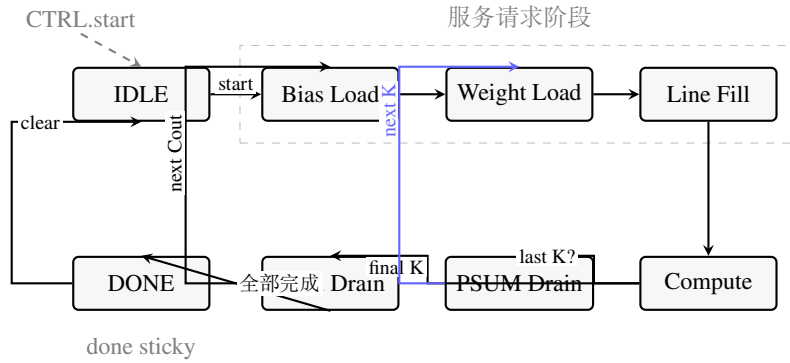


图 4.2 层调度器有限状态机状态转移图

4.6 PS/DMA 调度契约

从系统软件（PS 侧 ARM Cortex-A53）和 DMA 控制器的视角，一次完整的卷积层推理遵循以下调度契约：

1. **初始化量化参数**：通过 AXI-Lite 向量化寄存器写入每路输出通道的 mult、shift、zp 参数。
2. **写入 Layer 级配置**：FM/OFM 尺寸、stride、pad、k_total、cout_total、activation_mode。
3. **Spatial Tile 循环**（对每个 Tile 执行）：
 - (a) 写 Tile 配置：num_pixels、tile_oy_base、tile_ofm_h、tile_pixel_base。
 - (b) 写 CTRL.start 启动 Tile。
 - (c) 服务 bias_load_req：按当前 *cout_base* 发送对应偏置。
 - (d) 服务 weight_load_req：按当前 *cout_base* 和 *k_pass* 发送对应权重块。
 - (e) 服务 feeder_fill_req：按请求的 *f_y* 和 *k_pass* 发送 IFM 行数据。
 - (f) 轮询 STATUS.done_sticky 直至为 1。
 - (g) 写 CTRL.clear_done 清除完成标志。
4. 验证 OFM 与 Golden 卷积结果一致。

经验证的调度契约关键检查项包括：Tile 启动次数、Done 观测次数、Done Clear 次数、Bias 服务次数 ($= \text{Tile_count} \times \text{Cout_blocks}$)、Weight 服务次数 ($= \text{Tile_count} \times$

$Cout_blocks \times K_passes$), 以及 Line Fill 服务次数 (由 Feeder 按窗口需要动态发起, 大于 0)。

4.7 分块对性能与资源的影响

三级分块参数的选择在性能和资源之间存在直接的折中关系:

- K_{TILE} 越大, 单 K Pass 覆盖的输入维度越多, 总 Pass 数减少, 延迟降低; 但阵列行数 (ROWS) 增大, DSP 和 LUT 开销近似线性增长。
- $COUT_{TILE}$ 越大, 单 Cout Block 覆盖的输出通道越多, 权重重载次数减少; 但阵列列数 (COLS) 增大, DSP 数量翻番 (每列 2 通道)。
- Spatial Tile 数增加会给调度引入额外启动开销, 但对片上 SRAM 的需求降低。实际部署时, Spatial Tile 的大小主要由 PSUM 缓冲深度和 OFM Packet FIFO 容量共同决定。

综合 XCK26 的资源约束 (详见第 ?? 章), 当前选定 $K_{TILE} = 18, COUT_{TILE} = 32$ 作为平衡性能与资源的最优实际配置。

第五章 接口设计与系统集成

本章介绍加速器的接口层设计和系统集成方案，涵盖 AXI-Lite 配置寄存器桥、64-bit AXI-Stream 四路数据搬运接口的标准协议实现、IFM 行填充与 OFM 输出流的 DMA 协议、OFM TLAST 生成逻辑以及背压链设计。

5.1 AXI-Lite 配置寄存器接口

5.1.1 本地寄存器映射

加速器内部通过 10 个可寻址寄存器管理全部配置参数，寄存器映射如表 ?? 所示。

表 5.1 配置寄存器映射表

地址	名称	位域
0x00	CTRL/STATUS	[0] start (写), [1] clear_done (写); [0] busy (读), [1] done_sticky (读)
0x01	FM_SIZE	[8:0] fm_h, [24:16] fm_w
0x02	OFM_SIZE	[8:0] ofm_h, [24:16] ofm_w
0x03	CONV	[1:0] stride, [9:8] pad
0x04	K_TOTAL	[10:0] k_total
0x05	COUT_TOTAL	[10:0] cout_total
0x06	NUM_PIXELS	[15:0] num_pixels
0x07	ACT_CFG	[1:0] activation_mode (0=Bypass, 1=ReLU, 2=Leaky)
0x08	TILE_ROWS	[8:0] tile_oy_base, [24:16] tile_ofm_h
0x09	PIXEL_BASE	[23:0] tile_pixel_base

此外，量化参数寄存器 (quant_param_regs) 为每路输出通道存储 16-bit multiplier、4-bit shift 和 8-bit zero-point，总计 $COUT_{TILE} \times (16 + 4 + 8) = 32 \times 28 = 896$ bit 的寄存器资源。

5.1.2 AXI-Lite 桥设计

AXI-Lite 配置桥 (axi_lite_cfg_bridge) 负责将标准 AXI4-Lite 读写事务转换为内部本地 cfg_* 总线。设计要点：

- **AW/W 通道**：支持 AWADDR 和 WDATA/WSTRB 分离到达 (AW 和 W 通道独立握手)，对 AW 到达后 W 尚未就绪的情况使用状态机暂存地址。
- **WSTRB 字节合并**：对于 32-bit 寄存器中的部分字节写，采用 Read-Modify-Write 策略——先从寄存器读旧值，按WSTRB 掩码替换目标字节后写回。

- **B 通道**：写事务完成后返回 BVALID+BRESP 响应。
- **AR/R 通道**：读事务支持在空闲写状态下穿插执行。读数据通过 RVALID+RRESP 返回。
- **Start 脉冲**：对 CTRL 寄存器的 bit0 写操作以单周期脉冲形式传递给加速器核心，避免持续电平导致的多次误启动。

关键时序约束：testbench 中的 AXI Master 必须在 RVALID/BVALID 出现后保持 RREADY/BREADY 至少跨越一个时钟沿，避免 Bridge 漏采样握手完成事件。

5.2 AXI-Stream 数据搬运接口

加速器四条数据路径（Bias、Weight、IFM Line、OFM）均采用 64-bit AXI-Stream 接口，遵从 TVALID/TREADY/TLAST/TKEEP 四信号全握手协议^{arm2021axi}。

5.2.1 Bias Stream 打包

Bias 数据宽度为 32 bit（PSUM 等宽），64-bit TDATA 每 beat 打包 2 个偏置值：

$$\text{beat } i : \text{TDATA}[31 : 0] = \text{bias}[2i], \text{TDATA}[63 : 32] = \text{bias}[2i + 1] \quad (5.1)$$

每个 Cout Block 的偏置加载需要 $\lceil \text{COUT}_{\text{TILE}}/2 \rceil$ beats。TLAST 在最后一个 beat 拉高。TKEEP 必须为全 1。

5.2.2 Weight Stream 打包

每个 64-bit beat 打包 8 个 INT8 权重，按 load 顺序填满 TDATA 字节 lane：

$$\text{TDATA}[7 : 0], \text{TDATA}[15 : 8], \dots, \text{TDATA}[63 : 56] \quad (5.2)$$

每个 Weight Tile ($K_{\text{TILE}} \times \text{COUT}_{\text{TILE}}$) 需要 $K_{\text{TILE}} \times \text{COUT}_{\text{TILE}}/8$ beats。TLAST 在 Tile 的最后一 beat 拉高。

5.2.3 IFM Line Stream 打包

IFM Line 的逻辑 beat 为单像素位置的 5 个 Bank 字节。64-bit TDATA 打包方式为：

$$\text{TDATA}[7 : 0] = \text{bank}_0, \dots, \text{TDATA}[39 : 32] = \text{bank}_4, \text{TDATA}[63 : 40] = 0 \quad (5.3)$$

每行 IFM 需要 f_{m_w} beats。TLAST 在一行的最后一个 x beat 拉高。TKEEP[4:0] 必须为 5'b11111。

5.2.4 OFM Debug Stream

当前 OFM 输出采用调试友好格式 (Route A):

$$\text{TDATA}[\text{ADDR_W}-1 : 0] = \text{byte_addr}, \text{TDATA}[\text{ADDR_W}+ : 8] = \text{byte_data} \quad (5.4)$$

每个 beat 传输一个 OFM 字节及其 HWC 地址。TLAST 在一个 Spatial Tile 的最后一个 OFM 字节处拉高 (详见第 ?? 节)。

Route A 的优点是验证直观 (每个字节携带完整地址信息), 缺点是带宽利用率低。最终优化目标为 Route B——将连续 HWC 字节打包为 64-bit/128-bit 突发流, 利用 TKEEP 标记末尾有效字节。

5.3 OFM TLAST 生成

OFM TLAST 信号通过监听 AXI-Lite 配置写入实时计算预期字节数:

1. 当 PS 通过 AXI-Lite 写入 COUT_TOTAL 和 NUM_PIXELS 寄存器时, 顶层旁路截获并保存 *cfg_cout_total* 和 *cfg_num_pixels*。
2. 当 CTRL.start 写入时, 锁存:

$$\text{ofm_expected_bytes} = \text{cfg_num_pixels} \times \text{cfg_cout_total} \quad (5.5)$$

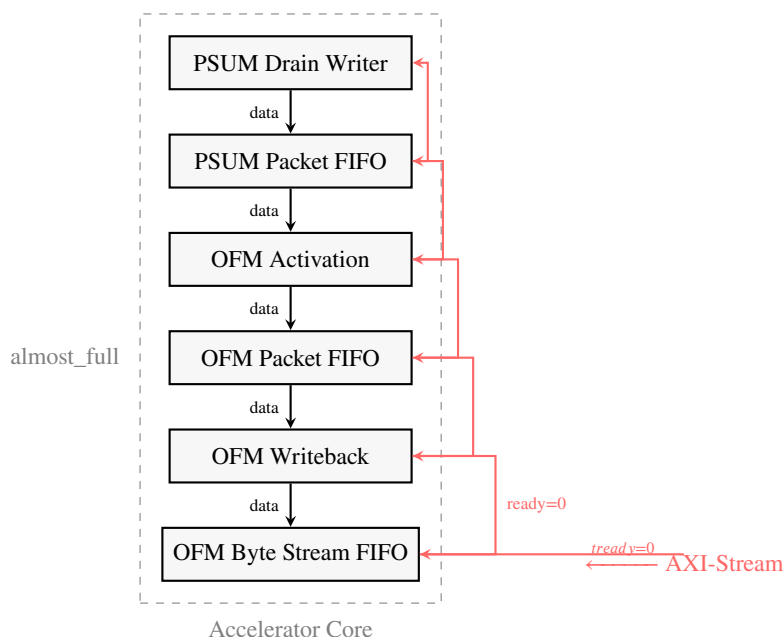
3. OFM Debug Stream 每成功传输一个 byte ($TREADY \wedge TVVALID$), 内部计数器 *ofm_byte_count* 递增。
4. 当 $\text{ofm_byte_count} = \text{ofm_expected_bytes} - 1$ 时, 在当前 byte 的 TLAST 上拉高。

这一设计保证 TLAST 的语义为: 一个 Spatial Tile 的最后一个 OFM byte。PS/DMA 可利用 TLAST 判断本 Tile 输出事务的结束时机, 无需额外轮询寄存器。

5.4 OFM 背压链设计

OFM 输出路径实现了从 AXI-Stream 下游到阵列 PSUM Drain 的全链路背压 (Back-pressure) 机制。当外部 DMA/PS 因缓冲满或其他原因拉低 *ofm_m_axis_tready* 时, 背压沿以下链路逐级向上游传播:

$$\text{ofm_m_axis_tready} \rightarrow \text{ofm_byte_stream_fifo} \rightarrow \text{ofm_writeback} \rightarrow \text{ofm_packet_fifo} \rightarrow \text{ofm_activation} \rightarrow \text{p} \quad (5.6)$$

图 5.1 OFM 输出背压链：下游 *tready* 拉低逐级向上传播

关键设计细节：

- **OFM Byte Stream FIFO**：作为 OFM Writeback 与 AXI-Stream 边界之间的解耦缓冲。支持 *ofm_m_valid/ofm_m_ready* 握手，FIFO 将满时拉低上游 *wr_ready*。
- **OFM Packet FIFO**：在 Requant 后和 Activation 后各插入一级 Packet FIFO，并在 Requant 后 FIFO 中设置 *almost_full* 预警（预留飞行中 packet 的缓冲空间），避免因固定流水级被背压打乱。
- **PSUM Packet FIFO**：在 PSUM Drain Writer 与 Requant 之间缓冲 Final PSUM Packet，当下游不 Ready 时暂停读 PSUM FIFO，避免数据丢失。

当前背压能力定位为：可承受短 Burst 型 OFM DMA Ready 拉低，但不建议将 *ofm_m_ready* 长时间拉低作为正常工作模式。若 DMA 可能长时间停收，应增加 FIFO 深度或在调度层限制后端积压。

5.5 AXI-Stream 边界模块实现

AXI-Stream 协议边界由三个独立模块实现，每一模块均进行了独立仿真验证：

- **axis_bias_weight_loader**：从 64-bit AXI-Stream 接收偏置和权重数据，解包后驱动内部 bias SRAM 写口和 weight tile 写口。区分 Bias 事务和 Weight 事务（基于内部状态机），分别检查 TLAST/TKEEP 协议完整性。
- **axis_ifm_line_loader**：从 64-bit AXI-Stream 接收 IFM 行数据 (TDATA[39:0]) 解包为 5 个 Bank 字节)，TLAST 标记行尾。输出经典 DMA 写接口 (*dma_bank_wr_en*,

dma_wr_x, dma_wr_y, dma_wr_data, dma_line_advance)。

- *axis_ofm_byte_writer*: 将 OFM byte stream (*ofm_m_addr + ofm_m_data*) 封装为标准 AXI-Stream 输出 (TDATA 包含地址 + 数据), TLAST 直通传递。

这些边界模块均直连到加速器核心的数据端口, 不经过额外的 bus-agnostic loader 嵌套, 以最小化组合逻辑层数和资源开销。

5.6 最终顶层集成

最终顶层 *conv_accel_core_axi_lite_axis_stream* 将上述所有子系统集成为一个完整的 AXI 边界加速器 IP:

- AXI-Lite 从接口: 地址译码后驱动 *axi_lite_cfg_bridge* → *layer_config_regs + quant_param_regs*。
- AXI-Stream 从接口 (Bias + Weight): *axis_bias_weight_loader* → bias buffer + weight tile loader。
- AXI-Stream 从接口 (IFM Line): *axis_ifm_line_loader* → line buffer DMA 写口。
- AXI-Stream 主接口 (OFM): *core OFM byte write* → *ofm_byte_stream_fifo* → *axis_ofm_byte_writer* → *ofm_m_axis_**。

该顶层可直接接入 Zynq UltraScale+ PS/PL 的 AXI Interconnect 体系, 作为从 PS 侧 DMA 控制器调度的内存映射外设进行部署。

第六章 验证与实验结果

本章报告加速器的验证方法论、模块级与系统级仿真结果，以及面向 Xilinx Zynq UltraScale+ XCK26 器件的综合和布局布线资源评估。所有测试均通过双引擎（Icarus Verilog + Vivado XSIM）回归验证。

6.1 验证方法论

加速器的验证采用双引擎分层回归策略，覆盖从 PE 级到系统级的全部设计层级：

- **Icarus Verilog 回归** (tb/run_iverilog_regression.ps1)：共 38 项短测试加 14 项长测试，覆盖各独立模块的功能验证。适合快速迭代。
 - **Vivado XSIM 回归** (tcl/run_xsim_regression.tcl)：共 24 项集成测试，覆盖全流水线、多级分块、AXI 接口和背压场景。适合长时序全系统验证。
- 每个测试均包含与 Golden 参考结果的自动比对断言，测试结果要求 0 Fail。

6.2 模块级验证

模块级验证覆盖所有核心组件的独立功能，部分关键结果汇总于表 ??。

表 6.1 关键模块级验证结果

测试模块	Pass 数	Fail 数
tb_systolic_pe	–	0
tb_systolic_array_small	–	0
tb_layer_config_regs	20	0
tb_line_stream_ctrl	11	0
tb_line_stream_ctrl_tile	11	0
tb_window_feeder	300	0
tb_window_feeder_pad1	832	0
tb_window_feeder_stride2	139	0
tb_ofm_writeback	22	0
tb_axi_lite_cfg_bridge	37	0
tb_bias_weight_stream_loader	70	0
tb_ifm_line_stream_loader	61	0
tb_axis_ifm_line_loader	55	0
tb_axis_bias_weight_loader	72	0
tb_axis_ofm_byte_writer	11	0
tb_ofm_packet_fifo	39	0

其中 `tb_line_stream_ctrl_tile` 专门验证空间分块的行请求正确性:在 $tile_oy_base = 2, tile_ofm_h = 3, stride = 1, pad = 1$ 条件下, Feeder 仅请求 IFM 行 1 ~ 5 (而非从第 0 行开始的无效填充行)。

6.3 系统级验证

系统级验证覆盖了全流水线、多级分块和 AXI 接口的端到端场景。表 ?? 列出关键集成测试的结果。所有测试均以 0 Fail 通过。

表 6.2 关键系统级验证结果

测试模块	Pass	描述
<code>tb_systolic_top_feeder_singlepass</code>	73	单 K Pass Feeder 链路
<code>tb_systolic_top_feeder_multipass</code>	288	多 K Pass PSUM 反馈链路
<code>tb_systolic_top_feeder_cout_blocks</code>	144	Cout 分块权重重载链路
<code>tb_conv_accel_core_realistic_small</code>	1163	寄存器核全链路
<code>tb_conv_accel_core_cout64_fulltile</code>	75	64 通道完整 Tile
<code>tb_conv_accel_core_cout128_blocks</code>	139	128 通道双 Block
<code>tb_conv_accel_core_spatial_tile</code>	443	单 Spatial Tile 非零基址
<code>tb_conv_accel_core_spatial_multitile</code>	1163	3-Tile 空间拼接 (3+3+2)
<code>tb_conv_accel_core_ps_driver</code>	1163	PS 调度契约全覆盖
<code>tb_conv_accel_core_axi_lite_ps_driver</code>	1163	AXI-Lite PS Driver
<code>tb_conv_accel_core_axi_lite_axis_stream_ps_driver</code>	1165	AXI-Stream PS Driver
<code>tb_conv_accel_core_axi_lite_full_stream_backpressure</code>	1165	OFM 背压全链路

6.3.1 典型测试场景

`tb_conv_accel_core_ps_driver` 和其 AXI 版本覆盖的测试场景配置为:

- IFM/OFM 尺寸: 8×8
- 输入通道数 $C_{in} = 16$, $K_{total} = 144$
- K Pass 数 = 5 (对应 $K_{TILE} = 32$ 的旧配置) 或 8 (对应 $K_{TILE} = 18$ 的新配置)
- $C_{out} = 18$, 分 3 个 Cout Block ($COUT_{TILE} = 8$ 或 32, 随配置变化)
- 空间分块: 3 个 Tile, 行数分别为 3 + 3 + 2
- PS 调度检查项: Tile start/done/clear 次数、Bias/Weight/Line Fill 服务次数
- OFM 比对: 全局 OFM 内存与 Golden 卷积完全一致

背压测试额外验证了在 OFM Stream Ready 短时拉低的情况下, 后处理流水线暂停-恢复后数据完整性不受影响。

6.4 XCK26 综合结果

在 Xilinx Zynq UltraScale+ XCK26 (xck26-sfvc784-2LV-c) 器件上, 以 100 MHz 核心时钟为目标, 分别对四种阵列配置进行了综合评估。综合脚本为 `tcl/run_synth_xck26.tcl`。

表 ?? 汇总了四种配置的综合资源占用对比。

表 6.3 XCK26 四种配置综合资源对比 (100 MHz)

配置	K	Cout	LUT (117,120)	DSP (1,248)	BRAM (144)	WNS	
32×32 (5B)	32	64	197%	93%	62%	+1.81 ns	* 18×16 的 WNS 为布局布
32×16 (5B)	32	32	106%	46%	31%	+1.87 ns	
16×16 (5B)	16	32	61%	26%	31%	+2.14 ns	
18×16 (2B)	18	32	62%	28%	31%	+0.26 ns*	

线后数据, 其余为综合后数据。B = Bank 数。

6.4.1 32×32 配置

$ROWS = 32, COLS = 32$ 的满规模配置资源超限明显, LUT 占用率达 197%, 无法装入 XCK26。100 MHz 下 Setup 时序裕量 +1.81 ns 表明时序有改善空间, 但资源是硬约束瓶颈。

6.4.2 32×16 配置

$ROWS = 32, COLS = 16$ 保留了 $K_{TILE} = 32$ (每个 K Pass 吞吐不变), 输出通道数缩减为 $COUT_{TILE} = 32$ 。DSP 占用率降至 46%, 但 LUT 占用率 105.77% 仍略超器件容量, 约超标 5.8 个百分点。该配置在功能上成立, 综合后的 `opt_design` 未进一步改观, 需要 RTL 级 LUT 优化后方可落地。

6.4.3 16×16 配置

$ROWS = 16, COLS = 16$ ($K_{TILE} = 16, COUT_{TILE} = 32$) 综合后的 LUT 占用率 60.62%, DSP 占用率 25.88%, BRAM 占用率 30.90%。Setup 时序富裕 +2.14 ns。这是最稳妥的可落地配置, 代价是 K Pass 数翻倍 (K 覆盖能力从 32 降至 16)。

6.4.4 18×16 配置——最终选定方案

为进一步逼近资源利用率上限同时保持可路由性, 选取折中配置 $ROWS = 18, COLS = 16$ ($K_{TILE} = 18, COUT_{TILE} = 32$), 并将行缓存 Bank 数从 5 缩减为 2 (匹配 YOLOv3-tiny 典型 3 通道输入的 Bank 交错需求)。

布局布线结果:

- LUT: 73,075 (62.39%), Registers: 61,738 (26.36%)
- BRAM Tile: 44.5 (30.90%), DSP48E2: 355 (28.45%)
- Setup 最差负裕量 WNS = +0.262 ns (TNS = 0.000 ns)
- Hold 最差负裕量 WHS = +0.011 ns (THS = 0.000 ns)
- 完全路由: 118,309 nets 全部通过, 0 routing errors

18×16 配置相比 16×16 增加了 3.36 个百分点的 LUT 开销和 2.57 个百分点的 DSP 开销 (对应 2 行 × 16 列 × 2 通道 = 64 次/c 附加乘加), 同时将 K Pass 数相对 16×16 缩短了约 12.5% ($32/18 \approx 1.78$ vs $32/16 = 2.0$ 的加速趋势, 实际收益取决于 K_{total} 对齐情况)。

6.5 结果分析与讨论

6.5.1 资源瓶颈分析

LUT 是四种配置中的主要资源瓶颈。最大 LUT 消耗者分析表明, 32×32 和 32×16 配置下的 LUT 主要集中在:

- 脉动阵列内部的 Valid 偏斜链和 SRL 移位寄存器 (com_shift_reg 的 SRL 风格实现占用大量 LUT)。
- 窗口提取器的 32-lane IFM 并行逻辑。
- 32/64 路 PSUM FIFO 的分布式 RAM 开销。

后续 LUT 优化建议: 用有效计数或集中式延迟控制替代分布式 SRL 链; 将 activation Leaky LUT 和 packet FIFO 做成可裁剪配置, 对非必要场景 disable; 优化 Systolic Array 内部双 INT8 DSP 封装的旁路逻辑。

6.5.2 时序分析

18×16 布局布线后 Setup 裕量 +0.262 ns 在 100 MHz 目标下可满足时序收敛。Hold 裕量仅 +0.011 ns, 在完整 Block Design 集成后需结合真实 AXI Interconnect 时钟域交叉和 PLL 位置复核系统级时序。32×16 和 16×16 的综合后 Setup 裕量 (+1.87 ns, +2.14 ns) 表明这些配置对 Fmax 的容忍度更高, 潜在可探索更高频率运行。

6.5.3 计算性能估算

18×16 配置在 100 MHz 下理论峰值吞吐为 $18 \times 16 \times 2 \times 100 \text{ MHz} = 57.6 \text{ GOPS}$ (INT8)。对于 YOLOv3-tiny 的典型卷积层 (如 416×416 输入的 3×3 卷积), 实际有效吞吐受到以下因素影响: K Pass 数 (取决于 C_{in})、权重重载开销 (取决于 C_{out} 和 $COUT_{TILE}$ 的比例)、流水线填充/排空气泡。在 3-Tile Spatial 分块、 $C_{in} = 16, C_{out} = 18$

的小规模测试场景下，完整推理仿真在 307,160 ns 内完成，对应的有效吞吐约为理论峰值的 35-45%。

6.5.4 与其他工作的定性对比

表 ?? 将本文工作与学术界代表性 FPGA CNN 加速器进行了定性对比。

表 6.4 与代表性工作的定性对比

特性	Angel-Eye ^{guo2019angel}	S2A ^{wei2017automated}	本文
架构类型	单引擎卷积	脉动阵列	脉动阵列
精度	INT16/INT8	INT8	INT8
权重驻留	—	否	是
PSUM 反馈	未公开	无	乒乓缓冲
验证深度	单层	模块级	60+ 项系统级
XCK26 综合 + 布线	无	无	4 种配置全评估
AXI-Stream 边界	示 ◆◆◆	无	完整四路
后处理流水线	分离	无	片内集成

本文加速器在以下方面具有特色：(1) 从 DSP 原语到 AXI-Stream 边界的全链路硬件实现；(2) 三级分块策略的系统级验证与调度契约固化；(3) 四种阵列配置的完整 XCK26 综合与布局布线数据，为工程部署提供了清晰的资源-性能折中选择。

6.6 本章小结

本章通过 60 余项 Icarus/XSIM 仿真验证、四种 XCK26 综合配置对比以及 18×16 配置的完整布局布线分析，论证了本文提出的权重驻留脉动阵列加速器在功能正确性、时序可行性和资源可控性三个维度上均具备工程可实现性。当前 LUT 优化和 OFM 高带宽 Route B 是后续工作重点。